

2-4-6 Task

System Documentation for Researchers

Group Hypothesis Testing — Online Experiment Infrastructure

Platform	Node.js web application + Qualtrics survey
Real-time layer	Socket.io (WebSocket connections)
Group size	3 participants per group (configurable)
Trial cap	20 rounds maximum per session
Data output	NDJSON session log + Qualtrics embedded data
Rule evaluation	Server-side only — participants never see the rule code

Prepared for internal researcher use. Do not distribute.

1. What This System Is

This document describes the technical architecture of a custom web application built to administer the **group 2-4-6 task** online. The 2-4-6 task is a classic paradigm in the reasoning literature (Wason, 1960) in which participants must discover a hidden rule by proposing number triples and receiving Yes/No feedback. The group version studied here requires participants to collectively propose triples, record individual rationales, and eventually announce what they believe the rule to be.

The system has two integrated components that work together: a **standalone web application** that handles everything during the interactive task itself (participant matching, real-time chat, triple submission, rule evaluation, and session logging), and a **Qualtrics survey** that handles everything surrounding the task (consent, instructions, comprehension checks, post-task questionnaires, and data export). Participants move seamlessly between the two via URL redirects.

The web application was built specifically for this experiment. It is not a commercial product and will require a researcher or technical collaborator to deploy and maintain it on a server.

Why a custom application rather than an existing tool?

The most widely used tool for real-time group experiments in Qualtrics is SMARTRIQS. After careful analysis, several limitations made SMARTRIQS unsuitable for this specific paradigm:

- SMARTRIQS uses a **linear survey flow** with no native looping. Running 20 iterative trial rounds would require manually duplicating blocks 20 times.
- The SMARTRIQS chat window is driven by **server-side state**, not by Qualtrics embedded data. This means the experimenter feedback (Yes/No verdict) cannot be injected into the live chat stream — participants would see it on a separate page instead.
- SMARTRIQS has no mechanism for requiring **consensus across participants** before a submission is accepted — a key feature of this study design.
- There is no way to collect **individual rationales** per trial round within SMARTRIQS's architecture.

The custom application solves all of these problems while offloading participant recruitment, survey questions, and data export to Qualtrics, where they are already handled well.

2. How a Session Works, End to End

The following describes a complete participant journey through the system, from survey start to completion.

Step 1	Qualtrics (pre-task)	The participant completes consent, instructions, and comprehension checks in Qualtrics. At the end of this block, Qualtrics redirects the participant to the web application. The redirect URL includes the participant's unique Qualtrics Response ID as a URL parameter (e.g., ?rid=R_abc123). This ID is the key that links the two datasets together.
Step 2	Waiting room	The participant arrives at the web application and is placed in a waiting room. The application monitors how many participants are waiting. As soon as the required number (currently 3) are present, a group is formed automatically and all participants advance to the task simultaneously. A pulsing indicator and live participant count keep the waiting participant informed.
Step 3	Task — discussion	The group sees a shared interface. The left panel shows a running history of every triple tested and its verdict. The right panel contains a live chat window where participants can communicate freely. The starting triple (2, 4, 6) and the instruction to discover the rule are shown at the top of the screen.
Step 4	Task — triple submission	When the group is ready to test a triple, each participant independently fills in: (a) the three numbers of the proposed triple, and (b) a personal written rationale explaining why they are proposing that triple. The rationale is private — participants cannot see each other's rationale. The triple must be the same across all participants before it is accepted.
Step 5	Consensus check	After all participants submit, the server checks whether all submitted triples are numerically identical. If they differ, all submissions are cleared and participants see a mismatch warning; they must discuss and resubmit. If they match, the server evaluates the triple against the hidden rule and broadcasts the verdict (Yes or No) to the whole group simultaneously.
Step 6	Rule announcement	At any point, the group can press 'Announce rule' to declare what they believe the rule to be. This opens a modal where they enter a free-text rule statement. On confirmation, the session ends for the entire group, the session data is written to disk, and all participants are redirected back to Qualtrics.
Step 7	Qualtrics (post-task)	Participants return to Qualtrics, which captures several summary parameters from the redirect URL (total trials, rule stated, group ID, etc.) and stores them as embedded data. Post-task questionnaires then collect individual-level measures. The Qualtrics dataset and the web application log are merged using the Qualtrics Response ID.

3. The Components of the System

3.1 The Rule Evaluator (server/ruleEvaluator.js)

This is the most experimentally important file. It contains the hidden rule that determines whether a proposed triple receives a 'Yes' or 'No' verdict. Because it runs on the server and is never sent to participants' browsers, participants cannot discover the rule by inspecting the page source.

The rule is defined in a single function called **conformsToRule(a, b, c)**. The current implementation is:

```
function conformsToRule(a, b, c) {  
    return b > a && c > b; // any strictly ascending sequence  
}
```

To change the rule for a different experimental condition, only this function needs to be modified. Nothing else in the codebase needs to change. The rule label (for researcher logs only) is defined in a separate constant called **RULE_LABEL** and is never transmitted to clients.

Important: changing the rule requires restarting the server. All in-progress sessions at the time of restart will be lost, because session state is held in memory (see Section 4 on data storage).

3.2 The Group Manager (server/groupManager.js)

This file manages all group state for the duration of a session. It handles the following responsibilities:

- **Waiting room:** Tracks participants who have arrived but not yet been matched. When enough participants are present (currently 3), they are assembled into a group.
- **Group state:** Stores the complete state of each active group in memory, including participant information, trial history with rationales, the chat log, and the current round number.
- **Consensus tracking:** For each round, records each participant's submitted triple and rationale. The `checkConsensus()` function compares all submissions numerically and returns one of three outcomes: 'waiting' (not everyone has submitted yet), 'mismatch' (all submitted but triples differ), or 'consensus' (all submitted the same triple, ready to evaluate).
- **Session export:** Packages the complete session record into a structured object for logging, including all trials, all rationales, the full chat log, and participant identifiers.

All group state is held in server memory (a JavaScript object). This means it does not survive a server restart. If the server crashes or is restarted mid-session, all in-progress sessions are lost. See Section 5 for how to address this.

3.3 The Server (server/index.js)

The server has two jobs: serving the web application files to participants' browsers, and handling all real-time communication between participants. Real-time communication is handled by Socket.io, a library that maintains a persistent connection between each participant's browser and the server, allowing messages to be sent and

received instantly without the page refreshing.

The server listens for the following events from participants and responds accordingly:

Event received	What the server does
join	Participant arrives. Added to waiting room. Group formed if enough participants are present.
chat_message	A participant sends a chat message. Server broadcasts it to the whole group immediately.
submit_triple	A participant submits a triple and rationale. Server runs consensus check. Responds with one of: <code>submission_received</code> , <code>submission_update</code> , <code>submission_mismatch</code> , or <code>trial_result</code> .
announce_rule	Group announces their rule. Server ends the session, logs the data, and sends each participant their redirect URL back to Qualtrics.
disconnect	A participant's connection is lost. Other group members are notified.

3.4 The Session Logger (server/logger.js)

When a group announces their rule and completes the task, the logger writes a complete session record to disk. Records are stored in a folder called **logs/** in newline-delimited JSON format (NDJSON) — one record per line, one file per day, named by date (e.g., `sessions_2025-03-15.ndjson`).

Each session record contains:

groupId	A unique identifier for this group's session. Used to link to Qualtrics data.
participants	Each participant's label (A/B/C), their Qualtrics Response ID, and whether they completed the session.
trials	Every trial in order: the triple, the verdict (Yes/No), a timestamp, and each participant's individual rationale.
chatLog	Every message sent in chat, with the sender's label, the message text, the round number, and a timestamp.
ruleAnnouncement	The group's stated rule, the round at which it was announced, and an automated flag indicating whether it appears correct.
status	Whether the session completed normally, or was terminated early.

3.5 The Participant Interface (public/)

Participants interact with a single web page that transitions between three states without ever reloading:

- **Waiting screen:** Shows a live count of how many participants have connected. Advances automatically when the group is full.
- **Task screen:** The main interface. Three regions: (1) a header bar showing the round number and participant label; (2) a feedback banner showing the verdict from the most recent trial; (3) a left panel with the full trial history; (4) a right panel with the live chat and submission form.
- **Completion screen:** A summary of the session (total trials, stated rule) with an automatic redirect back to Qualtrics after 5 seconds.

The submission form has two required fields: the three numbers of the proposed triple, and a text box for the participant's personal rationale. Neither field can be left blank. After submitting, the form locks and shows a contextual status message — blue if waiting for others to submit, amber if a mismatch occurred and the group needs to resubmit.

4. How Data Is Currently Stored

It is important to understand exactly when and how data is saved, because the current implementation has a significant limitation that affects data completeness.

4.1 When data is written to disk

Data is currently written to disk in **exactly one situation**: when a group successfully completes the task by pressing 'Announce rule' and submitting their rule statement. The `logSession()` function in `server/logger.js` is called once, at that moment, and writes the complete session record.

This means: if a group does not reach the announcement stage — because a participant drops out, hits the round cap without announcing, or closes their browser — NO data is saved for that session. The trial history, chat log, and rationales are all lost.

4.2 Where the data file lives

The log file is written to a folder called **logs/** on the server's local filesystem, in the same directory as the application. This has an important implication for cloud hosting: most low-cost cloud platforms (such as Railway, Render, or Heroku) use **ephemeral filesystems**, meaning the disk is wiped every time the application is redeployed or restarted. If you redeploy the server without first downloading your log files, the data is permanently lost.

Recommended practice: before any redeployment, download the current day's session log via the researcher endpoint: `GET /admin/sessions?key=yourAdminKey`. This returns the full log as a JSON array that can be saved to your local machine.

4.3 What the Qualtrics data contains

When participants return to Qualtrics after the task, the redirect URL carries several summary parameters that Qualtrics captures as embedded data fields. These are:

<code>task_complete</code>	Always '1' if the participant returned via the normal completion flow.
<code>participant_label</code>	The participant's label in the group: 'Participant A', 'B', or 'C'.
<code>total_trials</code>	The number of trials the group completed before announcing.
<code>rule_stated</code>	The group's stated rule (URL-encoded text).
<code>rule_correct_flag</code>	'1' if the stated rule contained ascending/increasing language; '0' otherwise. This is an automated heuristic and should be verified manually.

group_id

The UUID of the group. Use this to join the Qualtrics dataset with the session log.

The Qualtrics dataset and the session log are merged using **group_id** (present in both) or **qualtricsRid** (the Qualtrics Response ID, stored in the session log for each participant and also available natively in Qualtrics).

5. Current Shortcomings and Recommended Fixes

The current implementation is suitable for a small pilot study under controlled conditions but has several limitations that should be addressed before large-scale data collection. Each is described below along with a concrete resolution.

Shortcoming 1: Incomplete sessions produce no data

Problem: Data is written only when a group announces their rule. Any session that ends prematurely — due to participant dropout, hitting the round cap, or browser closure — generates no record at all. For a reasoning study, incomplete sessions may be scientifically meaningful: a group that never announces the rule is a different kind of result from a group that announces it incorrectly.

Resolution: Fix: Write a session record after every accepted trial (not just at announcement), and write an 'incomplete' record on disconnect. This requires calling `logSession()` in two additional places in `server/index.js`: inside the `trial_result` broadcast (after each round), and inside the disconnect handler (when a group is disrupted). The session status field already exists to distinguish 'complete' from 'incomplete'.

Effort: Low — approximately 10 lines of code in `server/index.js`.

Shortcoming 2: Session state does not survive server restarts

Problem: All group state — who is in which group, what round they are on, the trial history, pending submissions — is stored in the Node.js process's memory. If the server crashes, is restarted, or is redeployed while sessions are active, all in-progress sessions are irrecoverably lost.

Resolution: Fix: Replace in-memory state with a lightweight database. SQLite (via the 'better-sqlite3' npm package) would add persistence with minimal setup — no separate database server required, and the database is a single file on disk. For larger studies or multi-server deployments, PostgreSQL or Redis are more appropriate. This is the single most important architectural improvement before production use.

Effort: Medium — requires rewriting `groupManager.js` to read and write from a database instead of a JavaScript object.

Shortcoming 3: Log files are lost on server redeployment

Problem: The NDJSON log files are written to the server's local filesystem. On most cloud platforms, this filesystem is wiped on each deployment. A researcher who redeploys the server — even just to fix a bug — without first downloading the logs will lose all session data collected to that point.

Resolution: Fix: Either (a) use a persistent volume on your hosting platform (most platforms offer this as an add-on), (b) write logs to an external storage service such as AWS S3 or a database, or (c) establish a strict protocol of always downloading logs before any redeployment. Option (c) is the simplest and requires no code changes; options (a) and (b) provide stronger guarantees.

Effort: Low (protocol) to Medium (code change for S3 integration).

Shortcoming 4: No handling for participants who hit the round cap without announcing

Problem: When a group reaches the maximum number of trials (currently 20) without announcing a rule, the submission form is disabled and a message is shown in chat. However, the session is never formally closed, no data is written, and participants are left on the task screen with no clear path forward. They receive no redirect back to Qualtrics.

Resolution: Fix: When the round cap is reached, automatically trigger the `announce_rule` flow — either by forcing an announcement modal to open, or by closing the session server-side with a status of `'cap_reached'` and redirecting all participants.

Effort: Low — approximately 20 lines of code across `server/index.js` and `public/js/client.js`.

Shortcoming 5: No dropout recovery for remaining participants

Problem: If one participant drops out mid-session, the remaining participants are notified via a chat message but the session continues in a degraded state. The consensus check requires all active participants to submit, so the remaining participants can still proceed — but there is no bot substitution, no option to end the session gracefully, and no communication to remaining participants about what they should do.

Resolution: Fix: Add a timeout mechanism: if a participant does not submit within a configurable window (e.g., 3 minutes), they are marked as dropped and excluded from the consensus count. This is consistent with how SMARTRIQS handles dropouts. Alternatively, for a study where group composition is critical, terminate the session immediately on dropout and record it as incomplete.

Effort: Medium — requires a timeout system in `groupManager.js`.

Shortcoming 6: The rule-correctness check is a heuristic, not a true semantic comparison

Problem: When a group announces their rule, the server checks whether their stated rule contains keywords like 'ascending' or 'increasing'. This is a crude heuristic. A group that states 'numbers that go up' would be flagged as incorrect; a group that states 'ascending but only even numbers' would be flagged as correct even though it is wrong.

Resolution: Fix: The `rule_correct_flag` in the Qualtrics data should be treated as a preliminary filter only, not a definitive outcome variable. All rule announcements should be manually reviewed by the researcher. The stated rule is stored in full in both the session log and the Qualtrics embedded data, making this straightforward.

Effort: None (researcher review); or High if you want true semantic parsing.

6. Deploying the Application

The application is a standard Node.js web server. It can be run on any computer or cloud server that has Node.js installed (version 16 or later). Below are the steps to get it running.

6.1 Running locally (for testing)

To run the application on your own computer for testing:

```
npm install
npm start
```

The application will be available at <http://localhost:3000>. Open three browser tabs to simulate a group of three participants. Each tab will be treated as a separate participant and they will be matched automatically.

6.2 Configuring the application

Three settings can be adjusted without editing code, using environment variables set before starting the server:

PORT	The port the server listens on. Default: 3000.
QUALTRICS_RETURN_URL	The full URL of your Qualtrics survey. Participants are redirected here after the task, with summary parameters appended.
ADMIN_KEY	A password for the researcher data download endpoint. Default: 'researcher'. Change this before deployment.

```
PORT=3000
QUALTRICS_RETURN_URL=https://youruni.qualtrics.com/jfe/form/SV_xxxx
ADMIN_KEY=mysecretkey
npm start
```

6.3 Cloud deployment

For a live study, the application needs to be hosted on a server accessible via the internet with an HTTPS connection. (Qualtrics will not redirect to an HTTP URL in production.) Recommended platforms:

Railway (railway.app)	Simplest option. Connect a GitHub repository, set environment variables in the dashboard, and deploy with one click. Free tier available for pilots.
Render (render.com)	Similar to Railway. Offers a free tier with the caveat that the server sleeps after 15 minutes of inactivity — not suitable for a live study, but fine for testing.

DigitalOcean App Platform	More control, approximately \$5/month. Does not sleep. Suitable for a live study.
Institutional server	If your institution offers server hosting, this keeps all data within your institution's infrastructure, which may simplify IRB compliance.

7. Merging the Two Datasets

The study produces two separate datasets: the Qualtrics export (individual-level survey responses) and the web application session log (group-level task data). They are linked by two shared identifiers:

group_id	Present in both datasets. The Qualtrics embedded data field 'group_id' and the 'groupId' field in the session log contain the same UUID. Use this to join group-level task variables (total trials, trial history, chat log) to the Qualtrics data.
qualtricsRid	The Qualtrics Response ID (e.g., R_abc123) is passed to the web application on arrival and stored in the session log under each participant's record. It is also the native Responseld in the Qualtrics export. Use this for participant-level joins (e.g., linking individual rationales to survey responses).

Recommended merge procedure

- Export the Qualtrics data as CSV. The columns group_id and Responseld will be present.
- Download the session log via GET /admin/sessions?key=yourAdminKey. Save as JSON.
- Parse the session log into a flat table: one row per participant per trial, with columns for group_id, participant_label, round, triple, verdict, and rationale.
- Merge the two tables on group_id (for group-level variables) or on Responseld / qualtricsRid (for participant-level variables).

The session log stores rationales at the trial level, keyed by participant label (e.g., 'Participant A'). The Qualtrics data stores participant_label as an embedded data field. These two can be joined to link each rationale to the corresponding participant's survey responses.

8. File Structure Reference

The following files make up the application. Files marked with * are the ones most likely to require researcher attention.

server/ruleEvaluator.js *	Contains the hidden rule. Edit conformsToRule() to change experimental conditions. The rule label (RULE_LABEL) appears in researcher logs only.
server/groupManager.js	All group state and consensus logic. Change GROUP_SIZE (line 14) to adjust group size. Change MAX_ROUNDS (line 15) to adjust the trial cap.
server/index.js	The server and all Socket.io event handlers. This is where the complete interaction logic lives.
server/logger.js	Writes session records to disk. The LOG_DIR constant controls where files are written.
public/index.html	The participant-facing page. Contains all three screens (waiting, task, complete).
public/css/style.css	All visual styling. The CSS variable definitions at the top of the file control colours and typography.
public/js/client.js	All client-side logic: socket event handling, UI state transitions, form validation.
logs/ *	Created automatically. Contains daily NDJSON session log files. Download these regularly to avoid data loss on redeployment.
README.md	Technical setup instructions, environment variable reference, and Qualtrics integration guide.

End of document.